

跨平台多编译器差分测试

该课题的需求实际上面对的是不同运行时平台下的不同编译器二进制产物的运行状态分析。

目前差分测试基本均面对同一平台或同一编译器。

但是考虑到目前并没有严格的广泛自动化测试需求，该特点应该不会影响测试。

现有其他差分测试方法

基于二进制输出的差分测试基本遵循如下模式：

1. 生成（使用模糊生成、变异生成等方式）测试程序，其应当有确定的行为；
2. 用不同的编译器编译同一份源文件；
3. 比较产物输出结果；
4. 搜寻编译器崩溃、可接受性差异和运行时行为不统一等问题；
5. 根据出错的样例，构建一个最小化的剪枝代码。

Solidity 测试

考虑到 Solidity 并不是原生运行在裸机上的裸露二进制，其语义上的良性不需要额外的工作（和 C、C++ 等语言不同）。

如果选用 Solang 测试性提供的 EVM 产物，则可以转换为常见的基于良定义程序的测试，类似 Go、Java 等语言。

考虑到在区块链平台上构建可复现的稳定输出流相对困难^[1]，因此可能需要更多侧重于崩溃，包括运行时崩溃和编译器崩溃。

可能的思路

- 基础可用性，例如基本的字面简单合约。该测试项主要可以用于后续的回归检测。
- 多义性，例如部分高阶代码，可能缺乏完善的语义定义。
- 已知问题，很多已知问题实际上代表了一大类常见问题，因此作简单模仿变换就可以得到更多错误报告。
- 实验性/高阶特性，部分特性，尤其是复杂特性的组合，也很容易导致行为差异。

1. EVM 有确定的运行平台，但是 Solang 的平台不完全一致。同时以太坊和相对常见的开源虚拟机方案

没有提供理想的跨平台输入输出流。 ↩